

2. Remove Duplicates from Sorted Array

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**. Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length
```

```
int k = removeDuplicates(nums); // Calls your implementation
```

```
assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`

Output: `2, nums = [1,2,_]`

Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2 respectively.

It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,0,1,1,1,2,2,3,3,4]`

Output: `5, nums = [0,1,2,3,4,_,_,_,_,_]`

Explanation: Your function should return `k = 5`, with the first five elements of `nums` being 0, 1, 2, 3, and 4 respectively.

It does not matter what you leave beyond the returned `k` (hence they are underscores).

Constraints:

- $1 \leq \text{nums.length} \leq 3 \cdot 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- `nums` is sorted in **non-decreasing order**.

PROGRAM

```
class Solution {

    public int removeDuplicates(int[] nums) {

        int k = 1;
```

```

for (int i = 1; i < nums.length; i++) {

    if (nums[i] != nums[i - 1]) {

        nums[k] = nums[i];

        k++;

    }

}

return k;

}

}

```

OUTPUT

The screenshot displays a LeetCode submission for the problem "Remove Duplicates from Sorted Array". The submission is marked as "Accepted" with 362/362 testcases passed. The code is written in Java and implements a two-pointer approach to remove duplicates in-place.

Runtime Performance:

- Runtime: 1 ms | Beats 77.12%
- Memory: 46.38 MB | Beats 95.44%

Test Results:

- Accepted (Runtime: 0 ms)
- Case 1: Input: [1,1,2], Output: [1,2]
- Case 2: Input: [1,1,2], Output: [1,2]

The code snippet shown is:

```

class Solution {
    public int removeDuplicates(int[] nums) {
        int k = 1;

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[i - 1]) {
                nums[k] = nums[i];
                k++;
            }
        }

        return k;
    }
}

```